```
[inet_http_server]
port = *:9001
username = admin
password = admin123

[program:wsgi_server]
user=root
command=/bin/bash -c "source /etc/apache2/envvars && exec /usr/sbin/apache2

[program:metrics_collector]
user=root
command=python /app/metrics_collector.py
process_name=metrics_collector
autostart=true
autorestart=true
startretries=3
stopasgroup=true
stdout_logfile=/logs/metrics_collector.log
redirect_stderr=true

[program:metrics_publisher]
user=root
command=python /app/metrics_publisher.py
process_name=metrics_publisher
autostart=true
autorestart=true
startretries=3
stopasgroup=true
stdout_logfile=/logs/metrics_publisher.log
redirect_stderr=true
root@3cdb5a223fee:~#
```

Figure 1: Supervisord configuration file

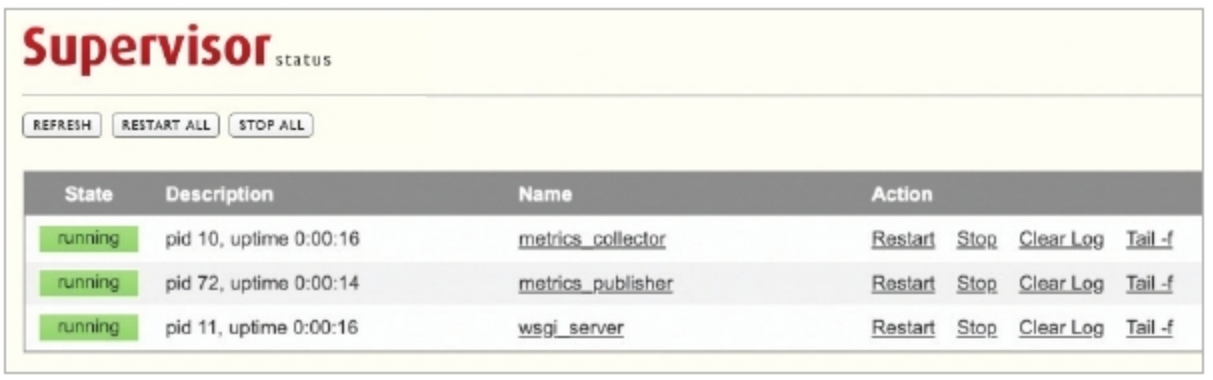

Figure 2: Supervisor web interface

applications in your local environment without connecting to a remote host or a virtual machine. It is used to deploy and scale the application in different environments such as a development to production environment. There are multiple advantages of using Docker Compose to manage a multi-service application.

We can have multiple isolated environments in a single host, can enable CI/CD for the environment, and can easily debug and identify the software bugs. We can build a specific service for an application and also get the benefits of using the Docker network and persistent volume to access and store the data layer of an application.

Let's assume we have a Dockerfile for all services created by *docker-compose*. Create a *docker-compose.yml* file and define the specifications for the application. Once the file is created, execute *docker-compose* in the same directory where the Docker Compose file exists. It will read the definition and interact with a Docker daemon to create the resources. It is recommended that you run it as a daemon process by passing the parameter *-d*.

In the example shown in Figure 3, I have defined three services. The first is the app tier, the second is the data tier and the third service is used to monitor the application's performance using Grafana, employing InfluxDB service as a data source.

Here, the app tier hosts different services such as the metrics monitor, the metrics collector and a WSGI

```
version: '3'
services:
    app:
        build:
          context: ./app
          dockerfile: Dockerfile
        volumes:
          - /datastore/app:/app
        ports:
            - "5000:5000"
            - "9001:9001"
            - "80:80"
        depends_on:
            - influxdb
    influxdb:
        image: influxdb
        volumes:
          - /datastore/influx:/var/lib/
        ports:
            - "8086:8086"
    grafana:
        build:
          context: ./grafana
          dockerfile: Dockerfile
        volumes:
          - /datastore/grafana:/var/lib
        ports:
            - "3000:3000"
```

Figure 3: Docker compose file

service. It is controlled and managed by Supervisord.

When an app container is started, Supervisord is started by the ENTRYPOINT script. Supervisord then starts the remaining process.

Docker Compose takes care of provisioning the services in order and maintaining the dependencies. It exposes the service port as per the definition. Persistent volume is mounted on the container for storing database files, Grafana dashboards and application code. Another benefit of using persistent volume for a development environment is that we do not need to build the service for every code change.

So if we are developing an application using Flask in a debug mode, it detects the changes in the source code and reruns the app without restarting. It avoids unnecessary builds and makes development much easier. We can directly use the repo path as a persistent volume mount path in a local environment. END

By: Radhakrishnan Ramakrishnan

The author is a technology enthusiast and DevOps engineer having expertise in Linux, OpenStack, AWS, Kubernetes, Web development and Hadoop administration.